

Case 9: Chained Requests — Create then Verify

Intermediate+

Create a post via POST, extract its id, then GET it back and verify the data matches.

New concept: Chaining multiple API calls in one test

API Endpoint

POST + GET `https://jsonplaceholder.typicode.com/posts`

What You Will Learn

- Chaining two API calls in one test function
- Extracting a value from one response to use in the next request
- End-to-end flow testing (create then read)
- f-strings to build dynamic URLs from response data

Study Plan

Step 1 — What is a chained request test?

Real-world tests rarely test one call in isolation. A common pattern is: create a resource, then immediately read it back to confirm it was saved correctly. This is an end-to-end flow.

Step 2 — Step 1: POST to create

Send a POST to /posts with a payload. Extract the id from the response: `post_id = response.json()['id']`. You will need this for the next call.

Step 3 — Step 2: GET using the new id

Use the extracted id to build the GET URL: `f'https://jsonplaceholder.typicode.com/posts/{post_id}'`. Then call `requests.get()` on it.

Step 4 — Compare the data

Assert that the id in the GET response matches the id you got from POST. Think: what type does the GET response return — a dict or a list? What operator compares two values?

Step 5 — Note on JSONPlaceholder

JSONPlaceholder is a fake API — the POST does not actually save data, so the GET will return the real post 101, not your created one. In a real API this chain would work perfectly.

Think Before You Code

Ask yourself these questions before writing a single line:

- After the POST, what type does `response.json()` return — a dict or a list?
- How do you extract the id from the POST response to use in the next URL?
- After the GET, do you need a loop or a direct key access to check the id?
- What would you name the two response variables to keep them clear?

Your Task

Write a pytest test that chains two API calls:

Step 1 — POST to /posts with payload:

```
{ title: 'Chain Test', body: 'Testing chains', userId: 1 }
```

Step 2 — Extract the id from the POST response

Step 3 — GET /posts/{id} using the extracted id

Before writing: think about these questions:

- What variable name will you use for each response?
- Is the POST response a dict or a list?
- Do you need a loop anywhere, or just direct key access?

The test must verify:

1. POST returns status 201
2. POST response contains an id
3. GET returns status 200
4. GET response id matches the id from POST

BONUS: Add a comment explaining why the title won't match on JSONPlaceholder and how you would assert it in a real API

Hint: `post_id = post_response.json()['id']` — then `f'{BASE_URL}/{post_id}'` for the GET. Two separate response variables: `post_response` and `get_response`.